LAMBDA & MAP()

A **lambda** is a small one-expression function. `map()` applies a function to **every** item.

1) Regular function vs lambda

Example :

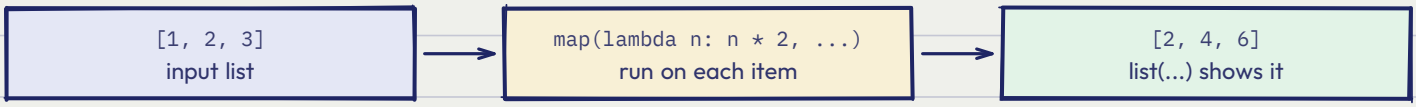
```
def double(number):
    return number * 2
double_lambda = lambda number: number * 2
print(double(5))
print(double_lambda(5))
add_five = lambda number: number + 5
is_even = lambda number: number % 2 == 0
print(add_five(10), is_even(8), is_even(7))
```

Output :

```
10
10
15 True False
```

a lambda has no name of its own and no return word: the expression is the result

2) map(): transform every item



Example :

```
numbers = [1, 2, 3, 4, 5]
doubled = list(map(lambda number: number * 2, numbers))
print(doubled)
marks_as_text = ["70", "85", "92"]
marks = list(map(int, marks_as_text))
print(marks, type(marks[0]))
bonus = list(map(lambda mark: mark + 10, [55, 68, 72, 80]))
print(bonus)
```

Output :

```
[2, 4, 6, 8, 10]
[70, 85, 92] <class 'int'>
[65, 78, 82, 90]
```

map gives back an iterator, so wrap it in list() to see every result at once

Quick check

`list(map(int, ["70", "85"]))` gives **[70, 85]**: map can take a built-in like `int` too.

FILTER() & REDUCE()

`filter()` keeps items that pass a test; `reduce()` folds a list into one value.

1) filter(): keep matching items

Example :

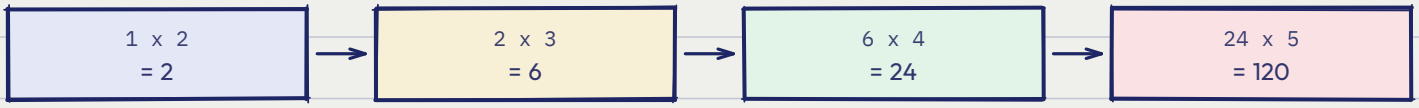
```
numbers = [1, 2, 3, 4, 5, 6]
print(list(filter(lambda n: n % 2 == 0, numbers)))
marks = [35, 72, 48, 90, 66]
print(list(filter(lambda mark: mark >= 50, marks)))
names = ["Asha", "Rahul", "Priya", "Amit", "Sanjay"]
print(list(filter(lambda name: len(name) > 4, names)))
```

Output :

```
[2, 4, 6]
[72, 90, 66]
['Rahul', 'Priya', 'Sanjay']
```

the lambda must answer True or False for each item, only the True items stay

2) reduce(): many values, one result



Example :

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
print(reduce(lambda t, n: t * n, numbers))
print(reduce(lambda t, n: t + n, [10, 20, 30, 40], 0))
words = ["cat", "elephant", "tiger", "giraffe"]
longest = lambda a, b: a if len(a) > len(b) else b
print(reduce(longest, words))
```

Output :

```
120
100
elephant
```

☆ REMEMBER

- `reduce()` lives in the `functools` module, so import it first.
- For a simple total, the built-in `sum()` is usually clearer.

COMPREHENSIONS

A **comprehension** builds a new list or dictionary in one readable line.

1) List comprehension

Example :

```
numbers = [1, 2, 3, 4, 5]
squares = [n * n for n in numbers]
even_squares = [n * n for n in numbers if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
raw = ["  Asha ", "Ravi  ", "  Meera"]
clean_names = [name.strip() for name in raw]
print(squares)
print(even_squares)
print(labels)
print(clean_names)
```

Output :

```
[1, 4, 9, 16, 25]
[4, 16]
['odd', 'even', 'odd', 'even', 'odd']
['Asha', 'Ravi', 'Meera']
```

Pattern	Form
basic	`[expression for item in iterable]`
with a condition	`[expression for item in iterable if condition]`
with if / else	`[a if condition else b for item in iterable]`

2) Dictionary comprehension

Example :

```
numbers = [1, 2, 3, 4]
print({n: n * n for n in numbers})
print({n: n * n for n in numbers if n % 2 == 0})
scores = {"Asha": 82, "Ravi": 75, "Meera": 91}
print({k: v for k, v in scores.items() if v >= 80})
prices = {"pen": 10, "notebook": 50, "bag": 800}
print({k: v * 0.9 for k, v in prices.items()})
```

Output :

```
{1: 1, 2: 4, 3: 9, 4: 16}
{2: 4, 4: 16}
{'Asha': 82, 'Meera': 91}
{'pen': 9.0, 'notebook': 45.0, 'bag': 720.0}
```

Practice & Revision

Write each one yourself first, then compare. Each solution is run for real.

Task 1 Lambda subtract.

Solution :

```
subtract = lambda a, b: a - b
print(subtract(10, 4))
```

```
6
```

Task 2 map: triple each number.

Solution :

```
print(list(map(lambda n: n * 3, [2, 4, 6])))
```

```
[6, 12, 18]
```

Task 3 filter: numbers over 10.

Solution :

```
numbers = [4, 12, 7, 18, 3, 15]
print(list(filter(lambda n: n > 10, numbers)))
```

```
[12, 18, 15]
```

Task 4 reduce: multiply the list.

Solution :

```
from functools import reduce
print(reduce(lambda a, b: a * b, [2, 3, 4]))
```

```
24
```

Task 5 Cubes of 1 to 4.

Solution :

```
print([n ** 3 for n in [1, 2, 3, 4]])
```

```
[1, 8, 27, 64]
```

Task 6 Names longer than 4 letters.

Solution :

```
names = ["Asha", "Rohan", "Mira", "Kabir"]
print([name for name in names if len(name) > 4])
```

```
['Rohan', 'Kabir']
```

Task 7 Pass or Retry labels.

Solution :

```
numbers = [1, 2, 3, 4]
print(["Pass" if n >= 3 else "Retry" for n in numbers])
```

```
['Retry', 'Retry', 'Pass', 'Pass']
```

Task 8 Passing marks as a dict.

Solution :

```
student_marks = {"Asha": 45, "Rohan": 78, "Mira": 88}
print({k: v for k, v in student_marks.items() if v >= 50})
```

```
{'Rohan': 78, 'Mira': 88}
```

Task 9 Prices up by 5 percent.

Solution :

```
prices = {"pen": 10, "book": 100}
new = {item: price * 1.05 for item, price in prices.items()}
print(new)
```

```
{'pen': 10.5, 'book': 105.0}
```

Task 10 Cube dictionary.

Solution :

```
print({n: n ** 3 for n in [1, 2, 3]})
```

```
{1: 1, 2: 8, 3: 27}
```

Quick comparison

Tool	Purpose	Example
`lambda`	a short function	`lambda n: n * 2`
`map()`	transform every item	`map(int, texts)`
`filter()`	keep matching items	`filter(is_even, nums)`
`reduce()`	combine into one result	`reduce(add, nums)`
`comprehension`	build a list or dict	`[n * n for n in nums]`